

RPS COHORTS • WORKSHOP NOTES • COHORT 02 • 19 SEPTEMBER 2026

Designing product journeys with AI. **Not just websites.**

Building a website with AI is easy. Building a consistent product flow with AI is not. These notes cover why AI output drifts on products, the three-layer design system it needs to stop drifting, and the live build: a Figma frame and a coded component library turned into working screens inside VS Code with Claude Code.



Cohort - 2

Designing
products with
AI ✨



THE SHORT VERSION

What this session was

Cohort 01 took one client brief to a live website with six prompts. Cohort 02 asked the harder question: what changes when you build a **product**, with many screens that must agree, every state, real data, and one drifted button that breaks trust across the app?

3

LAYERS A REAL
DESIGN SYSTEM
NEEDS

5

LIVE POLLS

10 min

FIGMA FRAME TO
WORKING SCREEN

85%

BEST TOOL SCORE
(CLAUDE CODE)

~6,000

UI UX JOB OPENINGS
TODAY

THE ARGUMENT

Websites are prompts & de-slop. **Products are prompts & systems.**

The models are already good. What they haven't seen is **your way of design**. So don't describe the design system in a document. Feed the AI the real artefacts: the coded component library, the Figma frame, screenshots and flows. Then make it **report blockers instead of inventing components**. Building that context is the real work, and prompting is the last step.

What we covered

- 01 Why AI output looks generic, and how to name design defects (slop, drift, hallucination) instead of vibes
- 02 What a product needs that a website doesn't: tokens, components, patterns, states, flows
- 03 Using your design system as a hard constraint, not a suggestion: the three token layers
- 04 The AI method: a one-time style guide vs an LLM-readable, audited design system
- 05 The AI-to-code pipeline: Figma → coded library → Claude Code in VS Code → localhost
- 06 Five tools tested on the same screen, Stripe's Protodash, and the Q&A

FIVE TAKEAWAYS

- **Context beats models.** The model is fine. Feed it your design.
- **Three layers or it's a style sheet:** primitive, semantic, component.
- **Feed code, not documents:** the coded library plus the Figma MCP link.
- **Always ask for blockers** so the AI never invents components.
- **Build context once, then journeys from screenshots.** Your UX judgement is now the bottleneck.



HOSTED BY

Vivin Richard

Principal Designer and Manager, RPS

Has mentored 10,000+ designers and builds AI-native design workflows for enterprise fintech at RPS. Thinks designers who learn to direct AI will outrun the ones who fear it.

CONTENTS

What is in here

The session in the order it happened: the ideas first, then the live build. Prompts from the demo are reconstructed from the recording, so change the details for your project but keep the guardrails.

01	Recap: Cohort 01 on one page	SIX STAGES, SIX PROMPTS
02	What AI can and cannot do	SLOP, DE-SLOP, THE GAP
03	The pipeline is shifting	HALLUCINATION & DRIFT
04	What makes a product a product	FIVE INGREDIENTS
05	Design systems, properly	THREE LAYERS
06	The AI method	TRADITIONAL VS LLM-READABLE
07	An IDE, for designers	VS CODE, EXTENSIONS, FOLDERS
08	Live demo: a Figma frame to a working screen	STEP BY STEP
09	Live demo: journeys from screenshots	THE PROMPTS
10	Tool shoot-out	FIVE TOOLS, ONE SCREEN
11	In the wild: Stripe's Protodash, and the recap	CASE STUDY
12	The tool landscape	EIGHT LAYERS
13	Questions from the room	SIX ANSWERS
14	Glossary, and what's next	REFERENCE

Where to find it in the recording

0:05	0:09	0:17	0:23	0:29	0:34	0:43
Recap of Cohort 01	What AI can't do, AI design	Current pipeline & the shift	Product ingredients, design systems	The AI method, agentic systems	What is an IDE	Live demo in VS Code
1:04	1:15	1:18	1:19	1:21	1:37	
Tool shoot-out	Stripe's Protodash	Recap of the demo	The tool landscape	Q&A	Cohort 03 & close	

RECAP · 0:05

01 Cohort 01 on one page

Last cohort took one client brief to a live website in under nine hours of work. There were six stages, each with its own prompt, and each document was the input to the next. Every prompt is in the Cohort 01 playbook PDF on the cohort website.

FIG 1 · THE COHORT 01 PIPELINE: BRIEF TO BUILD



Define. Gather requirements with a prompt, run a competitive study and a UX audit of any current site, and turn them into a report.

Structure & style. Write the content, then fix the design system. It was highlighted in the diagram because it matters most to the build.

Build. One build prompt that carries everything above into the page.

POLL
01

How many prompts does a deployable website need?

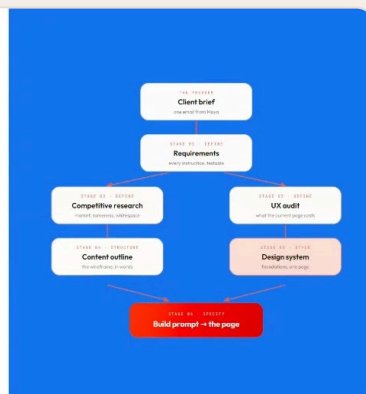
Options: style systems + build prompt · direct build prompt · six prompts · none.

ANSWER Six, one per stage. "Style systems + build prompt" was the most popular answer and isn't wrong, because those two do the heavy lifting.

THEN THE SHIFT

A website is **short-lived**: you design and ship it in two or three months. A product is a **long-term, consistent** system. Cohort 02 moves from websites to products, where prompts alone stop being enough.

From a client brief to a Shipped webpage



MISSED COHORT 01?

Log in to the cohort website: the recording and the playbook PDF are under past workshops. The PDF ends with an exercise. Do it before you try the product workflow in these notes, because the define and structure habits carry over.

[Claude](#)
[Vercel](#)
[WhatsApp group](#)

FOUNDATIONS · 0:09

02 What AI can and cannot do

The question in every room: will AI replace designers? No. ChatGPT has been out for about three and a half years, and there are still roughly 6,000 UI UX openings today. Companies need people who can **orchestrate** AI. The job is to upskill, not to worry.



Even behind this there is a programmer (human) who gives the measurements and the design

FIG 2 · CHISEL TO ROBOT ARM. EVEN BEHIND THE ROBOT THERE IS A PERSON GIVING IT THE MEASUREMENTS AND THE DESIGN.

WHAT IS AI DESIGN?

"AI product design integrates artificial intelligence into the design process to help enterprises build better products. The term can also apply to products developed using AI tools." (IBM, quoted in session)

Slop is generic, repetitive AI output that keeps making the same mistakes.

De-slop means removing it with proper prompting, a proper system and a manual review.

The real gap

WEBSITE DESIGN · PROMPTS & DE-SLOP

One page, one moment of attention

Visual taste carries the result

A spacing slip goes unnoticed

AI tools already do this well: just avoid AI slop and keep a simple design system

Six prompts and a reference

PRODUCT DESIGN · PROMPTS & SYSTEMS

Tokens, components, patterns

Every state: empty, error, first-run

Real data, permissions, edge cases

One drift breaks trust across the app

Many screens that must agree

POLL
02

What can AI do in building websites?

- YES** Create WebGL interfaces
- NO** Replace human designers completely
- NO** Empathise with users. It learns from the user data *you* gathered by empathising.

"WHAT MODEL DO YOU USE TO REDUCE GENERIC LAYOUTS?"

The model is fine. It hasn't seen your way of design.

Don't wait for a better model. Give the current one the right context.

PROCESS · 0:17

03 The pipeline is shifting

Leave the textbook design-thinking loop aside and look at a normal week: the designer builds in Figma, hands off, and the work passes through three more people before it goes live.

FIG 3 · PARTIALLY, OUR CURRENT PIPELINE: FOUR ROLES, FOUR PEOPLE



WHERE DESIGNERS ARE HEADING

Designers take on **01 and 02**. Figma is the master, and AI turns it into front-end UI code built on the real design system.

"Hand it to the developer and my job is done" was never the whole story. Now the move is from **Figma into front-end UI coding**. The U-turn looks hard but the process gets simpler, and developers are moving toward design from their side. **If you want a job in this industry, you need to be at this stage.**

TWO WORDS YOU'LL USE DAILY

Hallucination

The AI confidently invents a component, a colour or a fact. Usually it has lost context, for example when the context window fills up. "When AI lies confidently," as someone put it in chat.

Drift

The output slowly moves away from the system. Two buttons follow the design system and the third comes out a different colour.

SEEN IN A REAL REVIEW

A portfolio website had **three differently coloured buttons on a single page**. It lost consistency on one page, and nobody noticed for a long time. On a website that gets missed. **In a product it's a bug.**

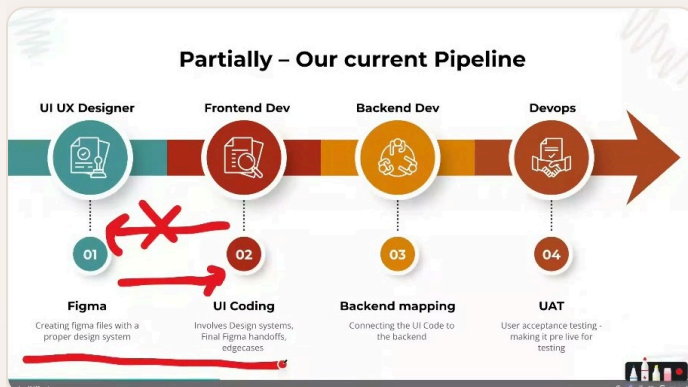


FIG 4 · LIVE ANNOTATION: DESIGNERS MOVE FROM FIGMA INTO UI CODING.

YOU OWN MORE

Figma and UI coding, up to a working front end on the real design system.

YOU CATCH YOUR OWN DRIFT

A wrong colour or spacing slip gets fixed by the person who designed it, before handoff.

DEVS START AHEAD

Developers receive working, on-system code, not a picture to rebuild over months.

FOUNDATIONS · 0:23

04 What makes a product a product

These are the five things AI needs to build a product properly. They are also the five things you need in Figma today. **If you can't hand them over, you can't prompt for them.**

01

Tokens

Named design decisions: colour, spacing, type, radius. Variables in Figma, CSS variables in code.

02

Components

Reusable blocks built only from tokens, such as buttons, inputs, chips and cards.

03

Patterns

How components behave together. Does the filter button open a modal or a dropdown?

04

States

Empty, error, first-run, loading, success. Every screen has more than one.

05

Flows

The step-by-step journeys that connect screens, derived from your UX work.

States, the part everyone skips

A first-time user on an e-commerce app opens the cart and finds nothing in it. That is an **empty state**, and it has to be designed. The same goes for error states, first-run states and many more. A website mostly has one state. A product has dozens per screen, and **each is somewhere the AI can drift** if it doesn't know the rules.

STATE	EXAMPLE
Empty	First login, empty cart, no quotes yet
First-run	Onboarding hints, sample data, first action
Error	Failed payment, invalid field, no network
Loading	Skeletons while data arrives
Success	Quote created, policy issued

TODAY'S REALITY CHECK

If you give a robot the wrong guidelines, how will it drill properly?

Are you using **variables** in Figma? **Components** and a real design system? The **same patterns** everywhere? **States** and properly derived **flows**?

If any answer is no, start there. Most people have only a shallow understanding of these terms, and without them you can't brief an AI.



Figma variables



Storybook

CHECKLIST · IS YOUR FIGMA FILE AI-READY?

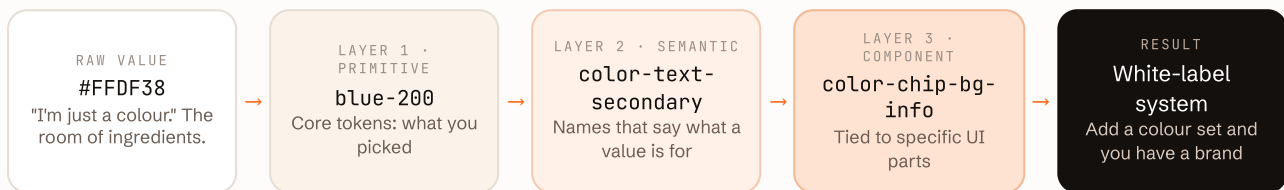
- Every colour, space and radius is a **variable**, not a hex value
- Components use **component-level tokens**, not raw semantic ones
- The design system is a **published library** and screens pull from it
- **Patterns** are written down: which trigger opens a modal and which a dropdown
- Empty, error and first-run **states** are designed for key screens
- **Flows** live in FigJam with the data points each screen needs

THREE LAYERS · 0:25

05 Design systems, properly

This is the most important thing to learn before you build products with AI. Think of baking: a big room of ingredients, the few you pick, the blocks you mix, and the cake that comes out.

FIG 5 · FROM RAW VALUE TO A WHITE-LABEL SYSTEM



THE LINE IN THE SAND

No three layers? Then it's a **style sheet**, not a design system.

Many teams, including senior designers at agencies, stop at the semantic layer and pull semantic tokens straight into components. There are **no component-level tokens**, so the system behaves like a UI kit.

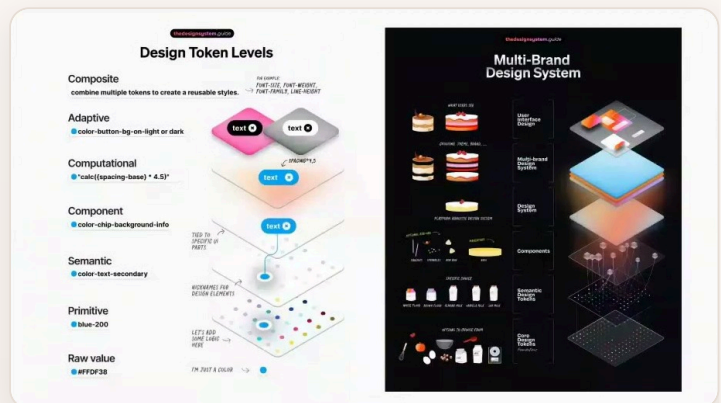


FIG 6 · DESIGN TOKEN LEVELS & MULTI-BRAND DESIGN SYSTEM. SOURCE: THEDESIGNSYSTEM.GUIDE

A complete system is **white-label**. Apply one colour set and it's one brand, apply another and it's a different brand. That is how multi-brand design systems work. Beyond the three core layers, mature systems add **computational** tokens ($\text{calc}(\text{spacing-base} \times 4.5)$), **adaptive** tokens (light or dark) and **composite** tokens (a full type style).

POLL
03

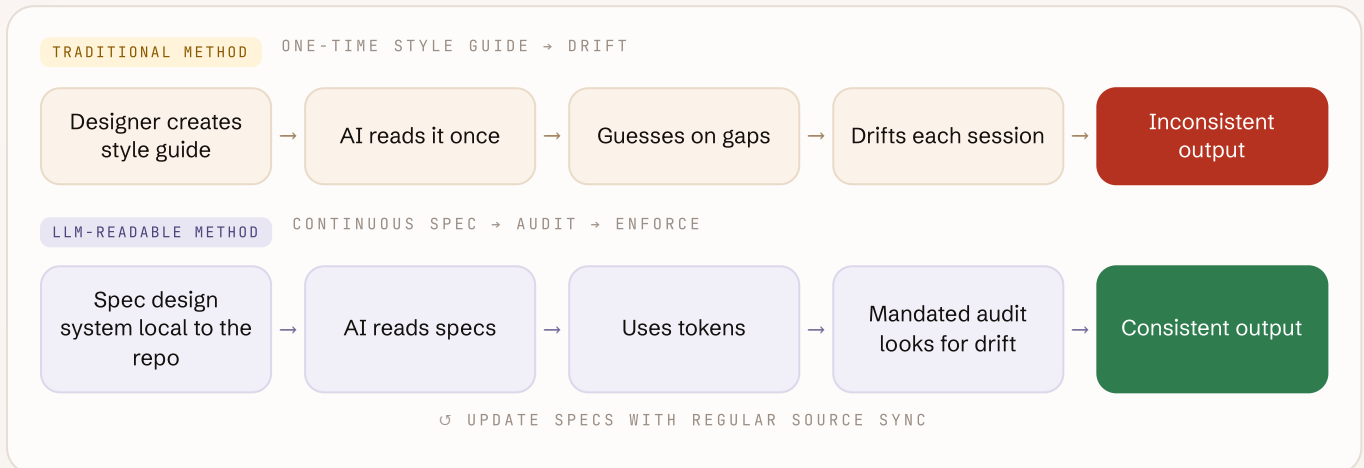
What are the three levels in a design system?

Most people answered **primitives**, **semantics**, **components**, and others answered **primitives**, **tokens**, **components**. **BOTH RIGHT** Tokenising is the semantic step.

THE AI METHOD · 0:29

06 Stop handing AI a style guide

Most people still make a style guide or UI kit, give it to the AI and say "read this once". The AI then guesses, and every new session loses what it knew. The fix is to make the design system **LLM-readable**.



What you feed in

- **Foundation:** tokens, components, brand, themes
- **Patterns:** how each component behaves and when to use which
- **The coded library:** ask your developers for your design system as a coded library (a zip). If you don't have one, start from a published open-source component library.

The AI reads it, understands it and uses the tokens from those files, so the output stays consistent. You will still see some drift, which is why the audit step exists. Vivin's tested consistency numbers are on page 11.

"CAN'T I JUST ASK AI TO BUILD MY DESIGN SYSTEM?"

It can. But can you **validate** that the token system it produced is right? Do you have an agentic pipeline with **guardrails** that trains the AI to do it properly? If not, use something that already exists: your developers' coded library, an agentic design system or a good published one.

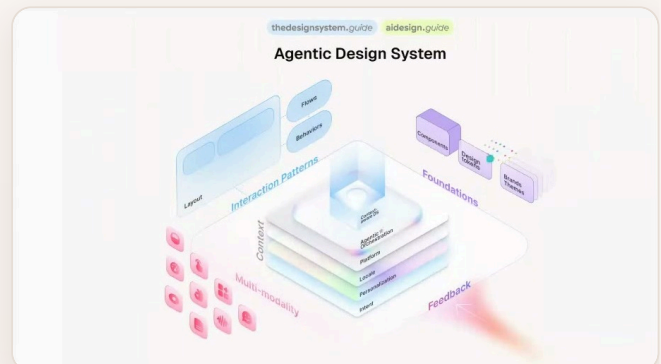


FIG 7 · AN AGENTIC DESIGN SYSTEM: FOUNDATIONS, PATTERNS, CONTEXT, FEEDBACK

POLL
04

What can agentic systems learn from?

ANSWER MCP servers, Markdown (.md) files and Skills files. Not only training data or only GitHub repos.

MCP (Model Context Protocol) builds context an AI can use, and Figma has its own MCP server. This is what the AI world is talking about right now.

MCP Figma MCP .md files Skills

VS CODE + CLAUDE CODE · 0:34

07 An IDE, for designers

"IDE" scares designers because it sounds like a coding environment. Take only the easy parts. Going from Notepad to an IDE is like going from Photoshop to Figma: a much faster place to work.

IDE · INTEGRATED DEVELOPMENT ENVIRONMENT

- It's no longer a Notepad editor
- It runs and tests your code instantly
- It's where developers write, debug and build software in one place
- It can connect many AI agents

POLL
05

Which of these are IDE tools?

ALL OF THE ABOVE with a nuance. **VS Code** and **Antigravity** are full IDEs. **Claude Code** and **Codex** are agentic tools, closer to partial IDEs.

HOW IDES WORK · THREE THINGS TO KNOW

- 01 Where you work: a local folder.** Create a folder and copy its path (the address bar on Windows, Get Info on a Mac). Open it in the IDE. That's why it's called *localhost*.
- 02 What you work with: extensions.** Claude, Codex and Figma live inside the IDE.
- 03 How you share it.** Zip the folder and send it to the dev team, or push it online.

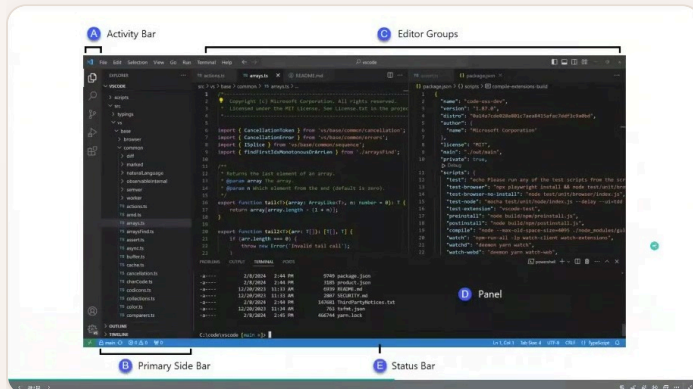
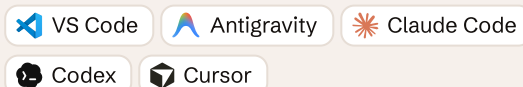


FIG 8 · VS CODE ANATOMY: ACTIVITY BAR, PRIMARY SIDE BAR, EDITOR GROUPS, PANEL, STATUS BAR

Things to remember

- localhost** Ask the agent to "host it locally" and it runs a preview on your machine.
- ▲ **Vercel** Your free live link. Tell it "push it to my Vercel" and share the link with a client.
- 🔗 **GitHub** An online repository. Store the project there instead of only locally.
- Point it** Open the project folder and point the agent at it: "this project lives here."



BEFORE THE DEMO · WHAT YOU'LL NEED



Everything shown uses **paid plans**: a professional Figma seat for Dev Mode, and a Claude plan. Cheaper alternatives exist, such as Codex, but in testing they weren't as accurate. See section 10.

LIVE DEMO · 0:43

08 A Figma frame to a working screen

The project was a real client flow: an insurance "Create a quote" journey. It had a published Figma design-system library, screens built from that library, and the same design system as a coded component library.

- 01 **Install two extensions.** In VS Code open Extensions, search for *Claude Code* and install it, then do the same for *Figma*. Sign in to each. The Claude icon opens a chat panel inside VS Code.
- 02 **Open a project folder** for the journey.
- 03 **Load the coded design system.** Get it from your developer as a zip, paste its path into the chat and say:

PROMPT · LOAD THE LIBRARY

Unwrap this component library and store it as part of this folder. Keep it as a memory. Do not hallucinate.

- 04 **Check the wiring.** Ask "Is Figma connected?" and it confirms the account. Ask "Is my design library ready to be used?" and it opens the packages and confirms both the code side and the Figma side.
- 05 **Copy the Figma context.** In Figma Dev Mode, select the frame (even a small container) and click *Copy example prompt*. That gives you a Figma MCP link to the exact node.
- 06 **Build, with guardrails:**

PROMPT · BUILD THE FIRST SCREEN

Implement this design from Figma. @[figma link]
Build it using the component library you just loaded.

Give me a blocker if you don't have any components.

If the task is done, localhost it.

WHY THE BLOCKER LINE

Without it, the AI gives you a thumbs-up and quietly **invents components**. You gave it 20 and it builds a 22nd, 23rd and 24th inside your screen without telling you.

WHAT HAPPENED · ~10 MIN

It screenshotted the Figma frame, **reviewed its output against the screenshot**, served it on localhost:5173 and reported **2 blockers** for components that weren't in the library.

TOKEN COST

Figma MCP estimated **46.8K** tokens for this frame and it used **27.8K**. A bigger screen was estimated at **90.3K**. So build the context once and don't repeat it for every screen.

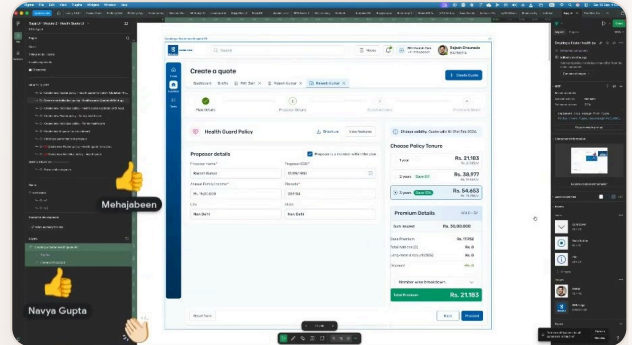


FIG 9 · THE SOURCE FIGMA FRAME, WITH THE DEV MODE MCP PANEL ON THE RIGHT

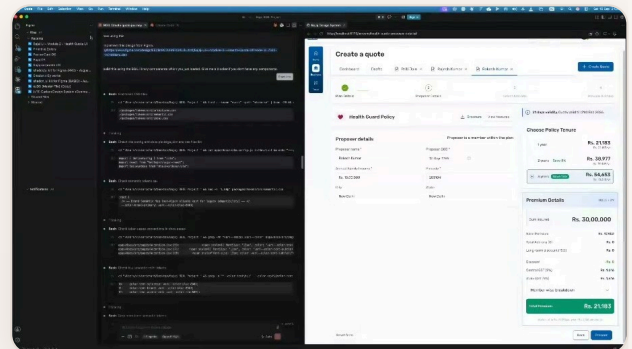


FIG 10 · CLAUDE CODE IN VS CODE (LEFT), THE RUNNING BUILD ON LOCALHOST:5173 (RIGHT)

Verdict from the room: "exact same." Not quite 100%: one icon differed, a key-value row component wasn't pulled, and a notes button had a blue border. These are small details and it's a good first output.

LIVE DEMO · 0:55

09 Journeys from screenshots

The first screen built the context: the Figma file, the zipped library and the guardrails. After that you don't need Figma for every screen. **A screenshot and a plain-English prompt are enough**, because the design system is already inside.

SCREEN 2 · SELECT ADD-ONS

Input: a screenshot of the next design (image.png), and a prompt in Vivin's own words, imperfect English and all.

PROMPT · NEXT STEP IN THE FLOW

When the user selects the Proceed button, create this in place of the Proposal details section, using the screenshot I've given you. Use the components (checkbox-styled input fields) exactly from the visual component library. I may not know the exact names in the library. Reference it and match it. For now, park the key-value pair and the closeable tab chip. I'll ask my dev team to add them to the library.

Result: a working step with **clickable, stateful components**, not a flat screenshot-to-code copy. There were small misses: the button hallucinated a little and one radio sat inside an input.

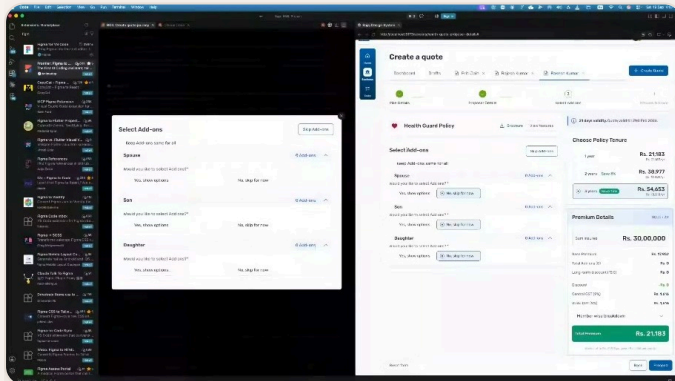


FIG 11 · SCREEN 2 BUILT FROM A SCREENSHOT, INSIDE THE SAME CODED DESIGN SYSTEM

SCREEN 3 · ADD-ONS FOR SPOUSE

Input: a **screenshot of ChatGPT research** listing add-ons such as maternity cover. It had no design system in it at all.

PROMPT · EXTEND THE JOURNEY

On selection of Yes, show options under Spouse. Use the radio button from our component library. For the selection-type field, use a checkbox from our component library instead of a radio button. Once add-ons are selected, show how many are selected in the chip component on top.

Result: it added the right options in the right way, with the count chip. In Figma this would have taken **at least 20 minutes**, and the chip logic is hard to do there at all.

THE NEW BOTTLENECK IS YOU

It has the design system and it knows the language. You can drop in a random screenshot and it builds in your system. So the question becomes **how good your UX is**: where each field goes, what spacing rule applies, which navigation and which flow.

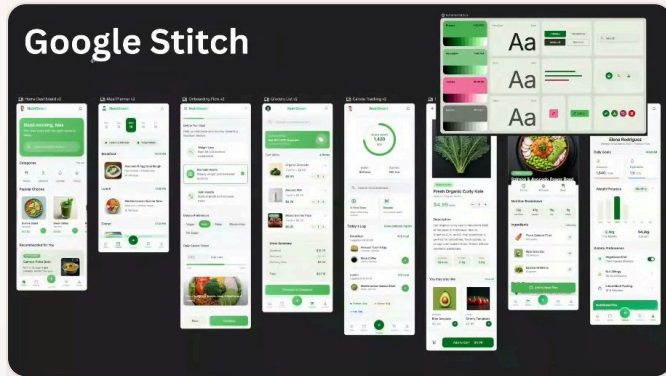
The RPS team built a full journey this way. It was shown as a 2.5-minute walkthrough video.

Recipe: build the context from one Figma screen. Then use your FigJam flows, architecture and data points to prompt with screenshots or user-flow diagrams. One good prompt can create several screens.

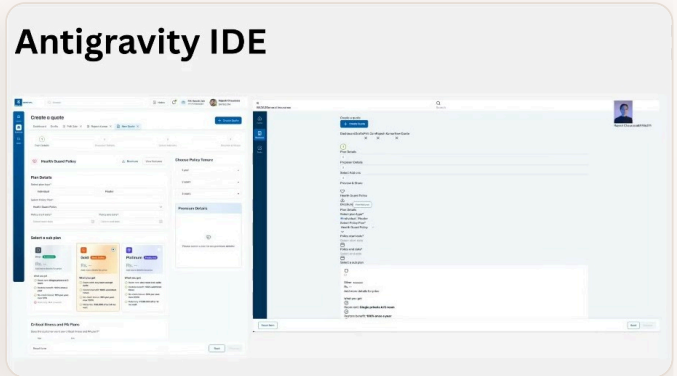
TOOL SHOOT-OUT · 1:04

10 Five tools, one product screen

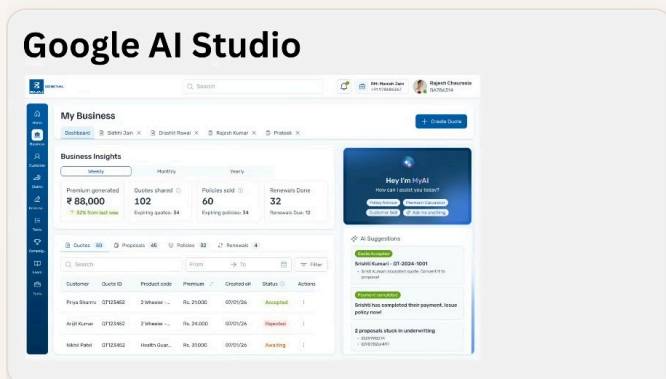
The same dashboard went through five tools. Colours and components gave most of them away. The percentages are Vivin's own scores from testing, not statistics.



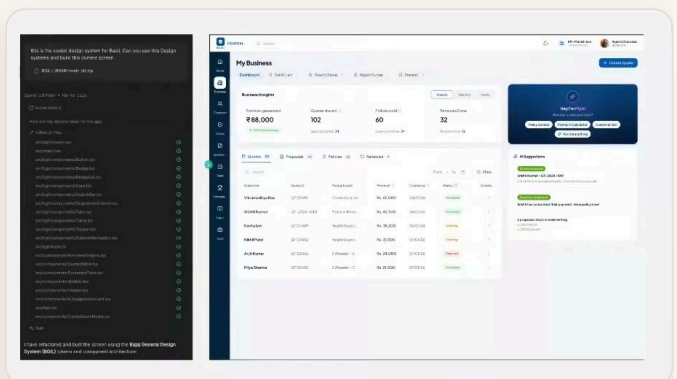
GOOGLE STITCH



GOOGLE ANTIGRAVITY



GOOGLE AI STUDIO



OPENAI CODEX, WITH THE CODED DESIGN SYSTEM

TOOL	WHAT HAPPENED	VERDICT
 Google Stitch	Nice-looking UI and it even generated a style guide, but the colours weren't consistent . Generic prompts give generic guidelines.	INCONSISTENT
 Antigravity	Given an AI-generated design system plus colours, it failed and looked like a Word document. A second try with the design system still missed.	LOST
 AI Studio	From an image it built its own components, a working search and realistic data . Then it hallucinated: a different-coloured side panel, odd icons and a lost container.	~50%
 Codex / GPT	With the Figma file as master it came close to Claude Code and is cheaper, though the side panel went wrong. It built a dropdown from a component screenshot and a kebab menu with Lucide icons.	~75%
 Claude Code	In VS Code with Figma MCP and the coded library (sections 08-09). It gave the most consistent output and reported blockers.	~85%

Debug tip from the Codex run: after the project folder was moved, Codex couldn't find it. Give it the new path and say "re-pull". With no remote to pull from, it set the project up as a local folder and carried on.

TOOL LANDSCAPE · 1:19

12 A drop in the ocean

What we learned today is one part of a much bigger pipeline. The industry is building bridges between intent, the design system, and code. This is the map shown in the session.

LAYER	TOOLS
Intent briefs, stories	Claude ChatPRD
System source of truth	Figma Variables Tokens Studio Storybook Supernova zeroheight Knapsack your npm package
Bridge agent access	Figma MCP Storybook MCP shadcn registry MCP Supernova MCP Stitch MCP CLAUDE.md · AGENTS.md · DESIGN.md Skills
Generate screens, flows	Claude Design Figma Make kits UXPin Merge AI Lovable Design Systems v0 + custom registry Google Stitch
Build real code	Claude Code Cursor OpenAI Codex Google Antigravity
Refine pixel control	Figma with agents writing to the canvas Paper
Verify before review	Storybook interaction & a11y tests Screenshot self-checks Token drift reports
Ship deploy	Vercel Lovable Replit

THE STACK FROM TODAY'S DEMO, END TO END



WHERE TODAY PUTS YOU

You've gone from **Figma designer to UI developer**. You have a live prototype you can test yourself and hand to a client. When a button is the wrong colour or the spacing is off, you can fix it yourself instead of filing it for a UI developer.

WHAT DEVELOPERS STILL OWN

Product-level launch, bug fixing and the backend. The difference is that they start from a working, on-system front end instead of spending months rebuilding it. The goal is **code as the single source of truth**, with the coded component library at its centre.

Q & A · 1 : 21

13 Questions from the room

Six questions from chat and the floor, answered in the session and condensed here.

Q1 · WHAT IS WebGL?

Web graphics: 3D inside a website. To get one, prompt "create a WebGL for my hero section." The RPS site hero is an example.



Q2 · FIGMA DS AND CODED DS: WHAT IF ONE IS MISSING?

With only the Figma design system it gets **really hard**. You'll get ChatGPT-grade output, and developers will have to go through it top to bottom updating components. That's fine if the client accepts it. If the client already has a component library, **ask for the zip** to get a consistent path.

Q3 · STRESS-TESTING COMPONENTS, KEEPING DESIGN AND CODE IN SYNC

Import the Figma file and the component files and Claude will **identify the drift**. Fixing it works partly, and fixing is where it hallucinates most. Figma and code almost never match 100%. RPS is building a tool that finds and fixes that drift and outputs one uniform library.

The direction is that Figma stays the master design file for the first screens, and **code becomes the single source of truth** after that.

Q4 · TECH STACK AND FOLDER STRUCTURE

Ask developers **up front** what folder structure they expect, and feed it to the AI. Match the framework to the coded library: if it's React, use React, and if the client has both React and Angular, ask which one. Agree on state management before you start.

Q5 · HOW DO I ADD A NEW COMPONENT?

Screenshot the component. Open the **component library alone** in its own folder, point Claude at it and say "using these tokens, build this component from the screenshot, without mistakes." Push it to Git, then pull it into the screens project.

Governance first: ask your lead if you really need it. Unused components pile up until the system is bloated and outdated.

Q6 · DOES AI ACTUALLY UNDERSTAND MY PROJECT?

No, you give it context. The first context was the Figma file, which told it the sidebar, top panel and three-column layout. It doesn't know your users. Do your UX thinking in a **separate tool** (FigJam, a ChatGPT or Claude project) and **never mix UX with the build journey**.

Treat the IDE as a builder that replaces Figma for screens. You could build 100 screens in 10-20 days, get client approval, and launch in four or five days.

WebGL WebGL React Angular Git GitHub FigJam ChatGPT

REFERENCE

14 Glossary

The vocabulary from the session. You'll need these words to name what's wrong with an output precisely, instead of saying "it doesn't feel right".

Slop	Generic, repetitive AI output	MCP	Model Context Protocol: gives an AI access to context in tools such as Figma
De-slop	Removing slop with prompting, a system and manual review	.md / Skills	Markdown instructions and skill files agents learn from
Hallucination	AI confidently inventing things, usually from lost context	IDE	Integrated Development Environment, e.g. VS Code
Drift	Output slowly leaving the design system	Extension	A plug-in inside the IDE, such as Claude Code or Figma
Primitive token	Named raw value, e.g. blue-200	Localhost	A preview running on your own machine
Semantic token	Named by purpose, e.g. color-text-secondary	Vercel	Free live links for clients
Component token	Tied to a UI part, e.g. color-chip-bg-info	Git / GitHub	Version control and online storage for the project
Style sheet	A "system" that stops before the component layer	Push / Pull	Send changes up to the repo / bring them down
White-label	A system that becomes any brand when you apply a colour set	Blocker	Something the AI must stop and report, not invent
Coded library	The design system as real components in code, usually a zip or npm package	Token burn	AI usage cost. Build the context once to save it
Context window	How much the AI can hold at once. When it's full, the AI starts to hallucinate	Dev Mode	Figma's developer view, where you copy the example prompt with the node link
Single source of truth	The one place that decides, increasingly the coded library	Governance	Deciding as a team whether a new component should exist at all

THE METHOD ON ONE LINE

Build the context once, from Figma and the coded library. Then build the journey with screenshots, flows and blockers.



THANK YOU

Now build a journey on your own product

Pick one screen you've already designed. Get the coded library from your developers, connect Figma and Claude Code in VS Code, and build that screen with the blocker rule on. Then prompt the next step from a screenshot. Share what you build and tag RPS.

THE PLAYBOOK

A free design guidelines PDF for everyone who stayed to the end.

THE REPLAY

The full workshop recording. It's sent after you fill in the feedback form.

THE Q&A COMPILATION

A curated FAQ from this session's chat, added to the handbook.

For Cohort 03, vote in chat or the community

- Do you want us to **review your portfolio**?
- Do you want a **top-up session on product design**, such as design systems in depth?
- Do you want us to **teach Claude Design**?
- Or put your own topic in the chat. This is the beginning of many sessions.



Join the RPS Cohorts WhatsApp community

It's free. The group hears first about new sessions, recordings and resources, and **internships and junior roles at RPS** will be posted there.